

VoyagerBot - Complete Documentation

VoyagerBot Project

November 2025

Table of Contents

VoyagerBot - Complete Documentation

An advanced autonomous robot control system built on Raspberry Pi 5

Version: 1.0

Date: November 2025

Project: Final Project for Semester VII 2025

Document Overview

This document consolidates all VoyagerBot documentation into a single comprehensive reference:

1. **Project Overview** - Introduction and key features
2. **Backend Documentation** - Python/FastAPI backend system
3. **Frontend Documentation** - Next.js/React web and mobile app
4. **Features Reference** - Detailed feature descriptions

Part 1: Project Overview

Introduction

VoyagerBot is an advanced autonomous robot control system built on Raspberry Pi 5, featuring real-time video streaming, AI-powered voice control, color-based object tracking, and autonomous navigation.

Key Technologies: - Backend: Python 3.9+, FastAPI, OpenCV, Socket.IO - Frontend: Next.js 14, React 18, TypeScript, Tailwind CSS - Hardware: Raspberry Pi 5, Camera Module, L298N Motor Driver - AI: Google Gemini for voice control

Quick Start

Backend Setup (Raspberry Pi)

```
cd voyagerbot/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 main.py
```

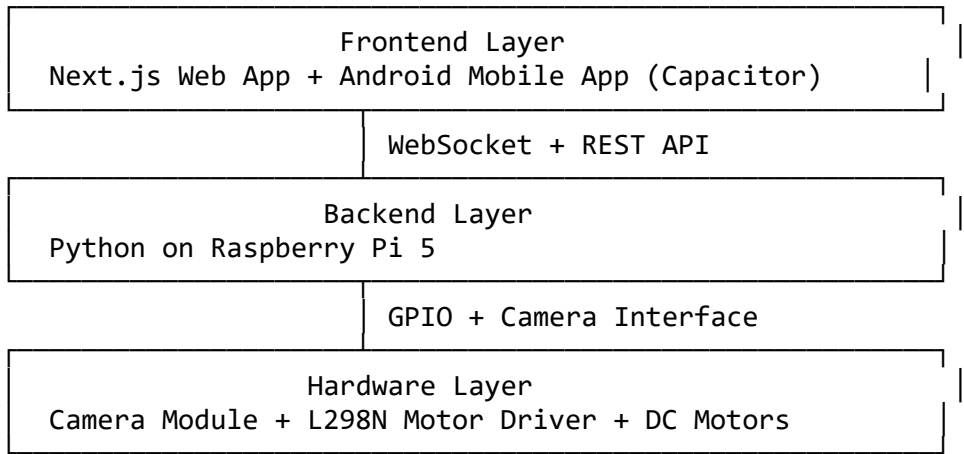
Server runs on: - REST API: `http://0.0.0.0:8000` - WebSocket: `ws://0.0.0.0:8765`

Frontend Setup

```
cd voyagerbot/frontend
npm install
npm run dev
```

Access at: `http://localhost:3000`

Architecture



Features Summary

Backend Features

- Real-time video streaming (15 FPS)
- Color-based object tracking (8 colors)
- Autonomous navigation with PID control
- Motor control via L298N driver
- System telemetry monitoring
- REST API and WebSocket server
- Safety features (watchdog, emergency stop)

Frontend Features

- Responsive web interface
- Live camera feed with overlay
- Joystick control
- AI voice assistant (Google Gemini)
- Telemetry dashboard
- Android mobile app
- Settings management

Part 2: Backend Documentation

Introduction

Table of Contents

1. Overview

- 2. [Technology Stack](#)
- 3. [Architecture](#)
- 4. [Installation & Setup](#)
- 5. [Configuration](#)
- 6. [Modules Documentation](#)
- 7. [API Reference](#)
- 8. [WebSocket Events](#)
- 9. [Hardware Integration](#)
- 10. [Development Guide](#)

Overview

VoyagerBot backend is a Python-based autonomous robot control system designed to run on Raspberry Pi 5. It provides real-time video streaming, color-based object tracking, autonomous navigation, and comprehensive telemetry monitoring. The backend uses a modular, multi-threaded architecture for optimal performance and maintainability.

Key Features

- **Real-time Video Streaming:** 15 FPS JPEG streaming via WebSocket
- **Color-based Object Tracking:** OpenCV-powered marker detection with 8 color options
- **Autonomous Navigation:** PID-controlled following with state machine
- **Motor Control:** Differential drive with PWM speed control
- **Telemetry Monitoring:** CPU, RAM, temperature, and FPS metrics
- **WebSocket Communication:** Real-time bidirectional communication
- **REST API:** Configuration and status endpoints
- **Hardware Abstraction:** Works with or without physical hardware

Technology Stack

Core Technologies

Technology	Version	Purpose
Python	3.9+	Primary programming language
FastAPI	>=0.104.0	Async web framework for REST API
Uvicorn	>=0.24.0	ASGI server for FastAPI
Socket.IO	>=5.10.0	WebSocket communication
OpenCV	>=4.8.0	Computer vision and image processing
NumPy	>=1.24.0	Numerical computing and array operations

Hardware Interface

Library	Version	Purpose
Picamera2	>=0.3.12	Raspberry Pi camera interface
RPi.GPIO	>=0.7.1	GPIO pin control for motors

System Monitoring

Library	Version	Purpose
---------	---------	---------

Library	Version	Purpose
psutil	<code>>=5.9.0</code>	System and process monitoring

Utilities

Library	Version	Purpose
python-dotenv	<code>>=1.0.0</code>	Environment variable management
aiohttp	<code>>=3.9.0</code>	Async HTTP client/server

Complete Dependencies

See backend/requirements.txt:

```
## Camera
picamera2>=0.3.12

## Computer Vision
opencv-python>=4.8.0
numpy>=1.24.0

## Web Server
fastapi>=0.104.0
uvicorn>=0.24.0
python-socketio>=5.10.0
aiohttp>=3.9.0

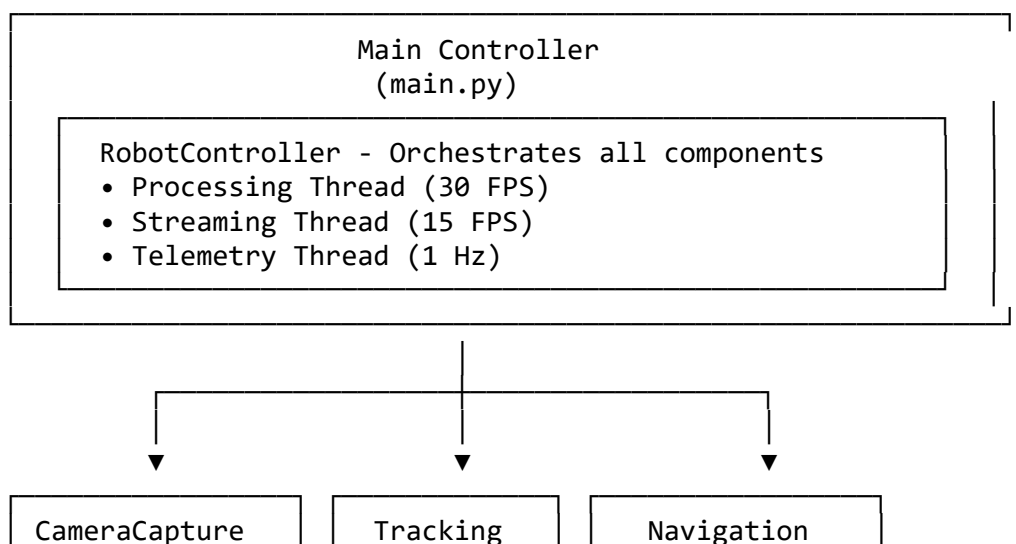
## GPIO Control
RPi.GPIO>=0.7.1

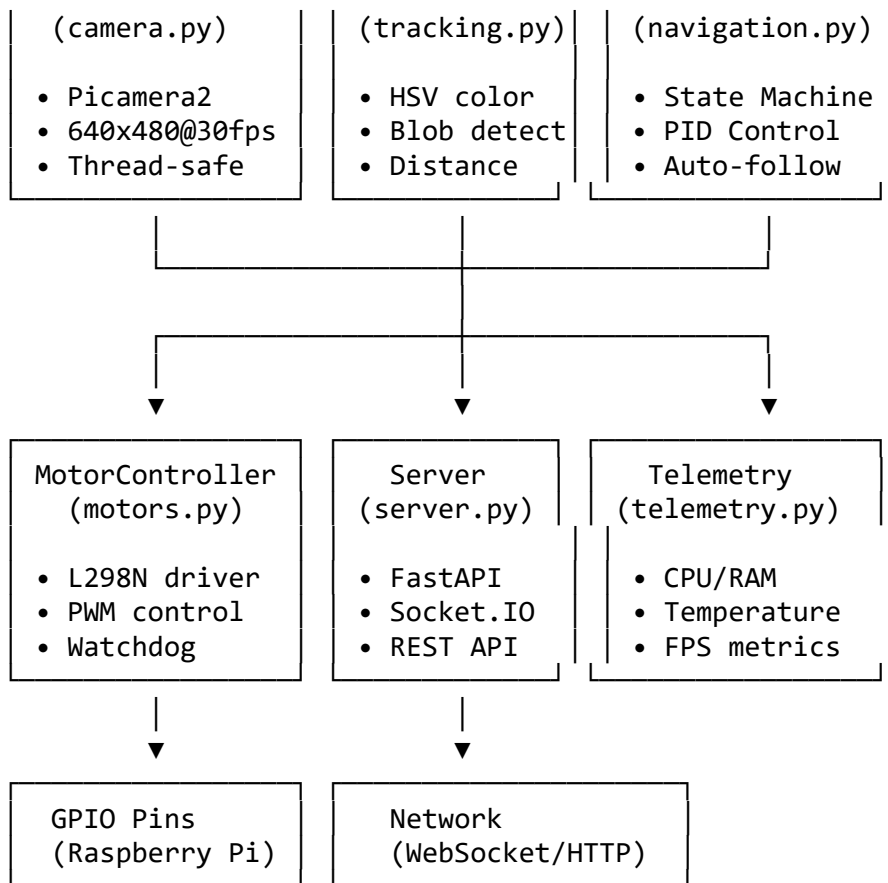
## System Monitoring
psutil>=5.9.0

## Utilities
python-dotenv>=1.0.0
```

Architecture

System Architecture Diagram





Thread Architecture

The backend uses three main processing threads:

1. **Processing Thread (30 FPS)**
 - Captures frames from camera
 - Runs color tracking detection
 - Updates navigation controller
 - Sends motor commands
 - Queues frames for streaming
2. **Streaming Thread (15 FPS)**
 - Retrieves frames from queue
 - Encodes to JPEG
 - Broadcasts via WebSocket
 - Manages streaming FPS
3. **Telemetry Thread (1 Hz)**
 - Collects system metrics
 - Monitors CPU/RAM/temperature
 - Tracks FPS counters
 - Broadcasts telemetry data

Data Flow

Camera → Processing Thread → Navigation → Motors
 Camera → Streaming Thread → WebSocket → Frontend

Installation & Setup

Prerequisites

- Raspberry Pi 5 (or 4 for testing)
- Raspberry Pi Camera Module v2 or v3
- L298N Motor Driver
- DC Motors (2x)
- Python 3.9 or higher
- Raspbian/Raspberry Pi OS (64-bit recommended)

Hardware Setup

1. Camera Connection

- Connect camera module to CSI port
- Enable camera interface: `sudo raspi-config` → Interface Options → Camera

2. Motor Driver Wiring

L298N → Raspberry Pi GPIO

Left Motor:

EN → GPIO 23 (Pin 16)
IN1 → GPIO 24 (Pin 18)
IN2 → GPIO 25 (Pin 22)

Right Motor:

EN → GPIO 17 (Pin 11)
IN1 → GPIO 27 (Pin 13)
IN2 → GPIO 22 (Pin 15)

Power:

12V → Motor power supply
5V → Raspberry Pi (if powering Pi from L298N)
GND → Common ground

Software Installation

1. Clone Repository

```
git clone https://github.com/aka-nahal/voyagerbot.git
cd voyagerbot/backend
```

2. Create Virtual Environment

```
python3 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Enable Camera (Raspberry Pi)

```
sudo raspi-config
# Navigate to Interface Options → Camera → Enable
sudo reboot
```

5. Test Camera

```
libcamera-hello # Should show camera preview
```

Running the Backend

1. Standard Mode (with hardware)

```
cd /home/runner/work/voyagerbot/voyagerbot/backend
python3 main.py
```

2. Development Mode (without hardware)

- The system automatically detects missing hardware
- Camera and motors will run in simulation mode
- Full functionality available for testing

3. View Logs

```
tail -f robot.log
```

Verification

After starting the backend, you should see:

```
INFO - Initializing modules...
INFO - Camera initialized successfully
INFO - Motor controller initialized (simulation mode)
INFO - Tracking initialized with color: orange
INFO - Navigation controller initialized
INFO - Telemetry monitor initialized
INFO - Communication server initialized
INFO - Starting server on http://0.0.0.0:8000
INFO - WebSocket server on ws://0.0.0.0:8765
INFO - All systems operational
```

Access points: - REST API: <http://<raspberrypi-ip>:8000> - WebSocket: <ws://<raspberrypi-ip>:8765> - Health Check: <http://<raspberrypi-ip>:8000/api/health>

.

Configuration

All configuration is centralized in `config.py`. No code changes needed for tuning.

Camera Settings

```
CAMERA_WIDTH = 640           # Capture resolution width
CAMERA_HEIGHT = 480          # Capture resolution height
CAMERA_FPS = 30              # Camera capture frame rate
STREAM_WIDTH = 480           # Streaming resolution width
STREAM_HEIGHT = 320          # Streaming resolution height
STREAM_FPS = 15              # WebSocket streaming FPS
JPEG_QUALITY = 65            # JPEG compression quality (1-100)
```

Tracking Settings

Color definitions (HSV ranges)

```
AVAILABLE_COLORS = {  
    'orange': {  
        'hsv_min': [5, 150, 150],  
        'hsv_max': [20, 255, 255],  
        'display_name': 'Orange',  
        'emoji': '🟠'  
    },  
    # ... 7 more colors (red, blue, green, yellow, purple, black, brown)  
}
```

DEFAULT_TRACKING_COLOR = 'orange'

MIN_BLOB_AREA = 500 *# Minimum valid marker size (pixels)*

MAX_BLOB_AREA = 50000 *# Maximum valid marker size (pixels)*

MARKER_LOST_TIMEOUT = 3.0 *# Seconds before marker considered lost*

Distance Estimation

TARGET_DISTANCE_CM = 125 *# Ideal following distance*

MIN_SAFE_DISTANCE_CM = 20 *# Stop threshold*

MAX_TRACKING_DISTANCE_CM = 500 *# Maximum detection range*

DISTANCE_CALIBRATION_FACTOR = 15000 *# Adjust based on calibration*

KNOWN_MARKER_WIDTH_CM = 15 *# Real-world marker width*

Motor Settings

GPIO Pins (match your wiring)

LEFT_EN = 23

LEFT_IN1 = 24

LEFT_IN2 = 25

RIGHT_EN = 17

RIGHT_IN1 = 27

RIGHT_IN2 = 22

Motor Parameters

PWM_FREQUENCY = 1000 *# PWM frequency in Hz*

MAX_SPEED = 100 *# Maximum duty cycle (%)*

MIN_OPERATIONAL_SPEED = 30 *# Minimum speed to overcome friction*

ACCELERATION_RATE = 5 *# Speed ramp rate*

AUTO_MAX_SPEED = 80 *# Speed limit in autonomous mode*

PID Tuning

Centering Controller (horizontal alignment)

PID_CENTER_KP = 0.8 *# Proportional gain*

PID_CENTER_KI = 0.0 *# Integral gain*

PID_CENTER_KD = 0.1 *# Derivative gain*

PID_CENTER_MAX_OUTPUT = 50 *# Max steering adjustment*

Distance Controller (forward/backward)

PID_DISTANCE_KP = 0.5

PID_DISTANCE_KI = 0.0

PID_DISTANCE_KD = 0.05

PID_DISTANCE_MAX_OUTPUT = 40

Navigation Settings

```
SEARCH_ROTATION_SPEED = 30      # Rotation speed when searching
CENTER_DEADZONE = 30            # Ignore small centering errors (pixels)
DISTANCE_DEADZONE = 10         # Ignore small distance errors (cm)
```

Network Settings

```
WEBSOCKET_HOST = "0.0.0.0"      # Bind to all interfaces
WEBSOCKET_PORT = 8765           # WebSocket port
API_HOST = "0.0.0.0"
API_PORT = 8000                 # REST API port
```

Safety Settings

```
WATCHDOG_TIMEOUT = 0.5         # Auto-stop if no command (seconds)
CONNECTION_TIMEOUT = 5.0       # Stop on disconnect (seconds)
THERMAL_LIMIT = 75.0           # CPU temperature limit (°C)
EMERGENCY_STOP_PRIORITY = True
```

Telemetry Settings

```
TELEMETRY_UPDATE_RATE = 1.0    # Update interval (seconds)
LOG_LEVEL = "INFO"             # DEBUG, INFO, WARNING, ERROR
LOG_FILE = "robot.log"
```

.

Modules Documentation

1. Camera Module (modules/camera.py)

Purpose: Manages Raspberry Pi camera capture with thread-safe frame access.

Class: CameraCapture

Initialization

```
camera = CameraCapture(width=640, height=480, fps=30)
camera.start()
```

Methods

Method	Description	Returns
start()	Initialize camera and start capture thread	None
get_frame()	Get latest frame (thread-safe)	numpy.ndarray or None
get_fps()	Get current capture FPS	float
stop()	Stop camera and cleanup	None

Features - Automatic reconnection on camera failures - Thread-safe frame buffering - FPS monitoring - RGB888 color format - Graceful error handling

Example Usage

```
from modules.camera import CameraCapture

## Initialize
camera = CameraCapture(640, 480, 30)
camera.start()
```

```

## Get frames
frame = camera.get_frame()
if frame is not None:
    print(f"Frame shape: {frame.shape}")
    print(f"Capture FPS: {camera.get_fps()}")

```

```

## Cleanup
camera.stop()

```

2. Color Tracking Module (modules/tracking.py)

Purpose: Detects and tracks colored objects using HSV color space.

Class: ColorTracker

Initialization

```
tracker = ColorTracker(color='orange')
```

Methods

Method	Parameters	Returns	Description
detect(frame)	frame: numpy.ndarray	dict	Detect marker in frame
set_color(color_name)	color_name: str	None	Change tracking color
get_current_color()	-	str	Get active color name

Detection Result Format

```

{
    'marker_detected': True/False,
    'position': {'x': 320, 'y': 240}, # Pixel coordinates
    'distance_cm': 45.5,             # Estimated distance
    'confidence': 0.85,              # Detection confidence (0-1)
    'blob_area': 5000,               # Marker area in pixels
    'timestamp': 1234567890.123
}

```

Detection Algorithm 1. Convert BGR to HSV color space 2. Apply color threshold (in-range) 3. Morphological operations (open + close) 4. Find contours 5. Filter by area (MIN_BLOB_AREA to MAX_BLOB_AREA) 6. Calculate centroid using moments 7. Estimate distance from blob area

Supported Colors - Orange ● - Red ● - Blue ● - Green ● - Yellow ● - Purple ● - Black ● - Brown ●

Example Usage

```
from modules.tracking import ColorTracker
```

```
tracker = ColorTracker('orange')
```

```

## Process frame
frame = camera.get_frame()
result = tracker.detect(frame)

```

```

if result['marker_detected']:
    print(f"Position: {result['position']}")
    print(f"Distance: {result['distance_cm']} cm")
    print(f"Confidence: {result['confidence']}")

## Change color
tracker.set_color('blue')

```

3. Motor Control Module (modules/motors.py)

Purpose: Controls DC motors via L298N H-bridge driver with PWM.

Class: MotorController

Initialization

```

motors = MotorController()
motors.start()

```

Methods

Method	Parameters	Returns	Description
set_speed(left, right)	left: int, right: int (-100 to 100)	None	Set motor speeds
stop()	-	None	Stop all motors
get_status()	-	dict	Get current motor state

Speed Format - Range: -100 to 100 - Positive: Forward - Negative: Backward - 0: Stop

Status Format

```

{
    'left_speed': 50,
    'right_speed': 50,
    'mode': 'manual',
    'timestamp': 1234567890.123
}

```

Features - Differential drive control - PWM speed control (1000 Hz) - Speed ramping for smooth acceleration - Watchdog timer (auto-stop on timeout) - Simulation mode (works without hardware) - Thread-safe operations

GPIO Pin Configuration

Left Motor: EN=23, IN1=24, IN2=25
 Right Motor: EN=17, IN1=27, IN2=22

Example Usage

```

from modules.motors import MotorController

motors = MotorController()

## Forward at 50% speed

```

```

motors.set_speed(50, 50)
time.sleep(2)

## Turn right (Left forward, right backward)
motors.set_speed(50, -50)
time.sleep(1)

## Stop
motors.stop()

.

```

4. Navigation Controller (modules/navigation.py)

Purpose: Implements autonomous navigation with state machine and PID control.

Class: NavigationController

Initialization

```
nav = NavigationController(motors, camera_width=640)
```

Methods

Method	Parameters	Returns	Description
update(tracking_data)	tracking_data: dict	None	Update navigation state
set_mode(mode)	mode: 'manual'/'auto'	None	Change navigation mode
emergency_stop()	-	None	Trigger emergency stop
get_state()	-	str	Get current state

Navigation Modes - manual: User controls motors directly - auto: Autonomous tracking and following

State Machine States 1. **IDLE**: Ready, waiting for mode change 2. **SEARCHING**: Rotating to find marker 3. **LOCKED**: Marker detected, preparing to follow 4. **FOLLOWING**: Actively tracking marker 5. **LOST**: Marker lost, attempting reacquisition 6. **STOPPED**: Emergency stop state

PID Controllers - **Centering PID**: Keeps marker centered horizontally - **Distance PID**: Maintains target distance

Zone-based Navigation - Left Zone: 0-40% of frame width - Center Zone: 40-60% of frame width - Right Zone: 60-100% of frame width

Example Usage

```

from modules.navigation import NavigationController

nav = NavigationController(motors, 640)

## Enable autonomous mode
nav.set_mode('auto')

## Update with tracking data
while True:
    frame = camera.get_frame()

```

```
tracking_data = tracker.detect(frame)
nav.update(tracking_data)
time.sleep(0.033) # ~30 Hz
```

5. Telemetry Monitor (modules/telemetry.py)

Purpose: Monitors system metrics (CPU, RAM, temperature, FPS).

Class: TelemetryMonitor

Initialization

```
telemetry = TelemetryMonitor()
```

Methods

Method	Returns	Description
collect_metrics()	dict	Collect all system metrics
get_cpu_temperature()	float	Read CPU temperature
record_processing_frame()	None	Track processing FPS
record_streaming_frame()	None	Track streaming FPS

Metrics Format

```
{
    'cpu_temp': 45.2,           # °C
    'cpu_usage': 25.5,         # %
    'ram_usage_mb': 512.3,     # MB used
    'ram_total_mb': 2048.0,    # MB total
    'ram_percent': 25.0,       # % used
    'fps_processing': 28.5,     # Processing FPS
    'fps_streaming': 14.2,     # Streaming FPS
    'uptime_seconds': 3600.0,   # System uptime
    'thermal_warning': False,   # CPU temp warning
    'network_bytes_sent': 1048576, # Total bytes sent
    'network_bytes_recv': 524288, # Total bytes received
    'timestamp': 1234567890.123
}
```

Example Usage

```
from modules.telemetry import TelemetryMonitor

telemetry = TelemetryMonitor()

## Collect metrics
metrics = telemetry.collect_metrics()
print(f"CPU: {metrics['cpu_usage']}%")
print(f"Temp: {metrics['cpu_temp']}°C")
print(f"RAM: {metrics['ram_usage_mb']} MB")
print(f"Processing FPS: {metrics['fps_processing']}")
```

6. Communication Server (modules/server.py)

Purpose: Provides WebSocket and REST API communication.

Class: CommunicationServer

Initialization

```
server = CommunicationServer()  
await server.start()
```

Features - FastAPI REST API - Socket.IO WebSocket server - CORS support - Client connection tracking - Async message processing

REST Endpoints: See [API Reference](#) section

WebSocket Events: See [WebSocket Events](#) section

API Reference

REST API Endpoints

Base URL: `http://<raspberrypi-ip>:8000`

GET /api/health

Health check endpoint.

Response

```
{  
  "status": "operational",  
  "uptime": 3600.5,  
  "timestamp": 1234567890.123  
}
```

GET /api/status

Get current system status.

Response

```
{  
  "navigation": {  
    "mode": "manual",  
    "state": "idle"  
  },  
  "tracking": {  
    "color": "orange",  
    "color_display": "Orange ●"  
  },  
  "clients_connected": 2,  
  "motors": {  
    "left_speed": 0,  
    "right_speed": 0  
  },  
}
```

```
    "telemetry": {
      "cpu_temp": 45.2,
      "cpu_usage": 25.5,
      "ram_usage_mb": 512.3
    }
  }
}
```

GET /api/config

Get all configuration parameters.

Response

```
{
  "camera": {
    "width": 640,
    "height": 480,
    "fps": 30,
    "stream_width": 480,
    "stream_height": 320,
    "stream_fps": 15
  },
  "tracking": {
    "current_color": "orange",
    "min_blob_area": 500,
    "max_blob_area": 50000,
    "available_colors": ["orange", "red", "blue", ...]
  },
  "motors": {
    "max_speed": 100,
    "auto_max_speed": 80,
    "pwm_frequency": 1000
  },
  "pid": {
    "center_kp": 0.8,
    "center_ki": 0.0,
    "center_kd": 0.1,
    "distance_kp": 0.5,
    "distance_ki": 0.0,
    "distance_kd": 0.05
  }
}
```

GET /api/colors

Get available tracking colors.

Response

```
{
  "colors": {
    "orange": {
      "hsv_min": [5, 150, 150],
      "hsv_max": [20, 255, 255],

```

```

        "display_name": "Orange",
        "emoji": "🟡"
    },
    "red": { ... },
    "blue": { ... }
},
"current": "orange"
}

```

POST /api/config

Update configuration (implementation required).

Request Body

```

{
  "camera_fps": 30,
  "stream_fps": 15,
  "pid_center_kp": 0.8
}

```

POST /api/calibrate/distance

Calibrate distance estimation.

Request Body

```

{
  "known_distance_cm": 50,
  "measured_blob_area": 5000
}

```

Response

```

{
  "status": "success",
  "new_calibration_factor": 15000,
  "message": "Distance calibration updated"
}

```

WebSocket Events

WebSocket URL: ws://<raspberrypi-ip>:8765

Client → Server Events

manual_control

Send manual movement commands.

Payload

```

{
  "direction": "forward", // forward, backward, left, right, stop

```



```
    "speed": 50 // 0-100
}
```

.

mode_change

Switch between manual and autonomous modes.

Payload

```
{
  "mode": "auto" // manual, auto
}
```

.

auto_command

Commands in autonomous mode.

Payload

```
{
  "command": "start" // start, stop
}
```

.

emergency_stop

Trigger immediate stop.

Payload

```
{
  "confirm": true
}
```

.

change_color

Change tracking color.

Payload

```
{
  "color": "blue" // orange, red, blue, green, yellow, purple, black, brown
}
```

.

settings_update

Update system settings.

Payload

```
{
  "setting": "pid_center_kp",
```

```
    "value": 0.9
}
```

Server → Client Events

connected

Connection confirmation.

Payload

```
{
  "message": "Connected to VoyagerBot",
  "client_id": "abc123",
  "timestamp": 1234567890.123
}
```

video_frame

JPEG-encoded video frame.

Payload

```
{
  "data": "base64_encoded_jpeg_string",
  "timestamp": 1234567890.123,
  "frame_number": 12345
}
```

tracking_data

Marker detection information.

Payload

```
{
  "marker_detected": true,
  "position": {"x": 320, "y": 240},
  "distance_cm": 45.5,
  "confidence": 0.85,
  "blob_area": 5000,
  "timestamp": 1234567890.123
}
```

motor_status

Current motor state.

Payload

```
{
  "left_speed": 50,
  "right_speed": 50,
}
```

```
    "mode": "manual",
    "state": "idle",
    "timestamp": 1234567890.123
}
```

.

telemetry

System metrics.

Payload

```
{
  "cpu_temp": 45.2,
  "cpu_usage": 25.5,
  "ram_usage_mb": 512.3,
  "ram_total_mb": 2048.0,
  "ram_percent": 25.0,
  "fps_processing": 28.5,
  "fps_streaming": 14.2,
  "uptime_seconds": 3600.0,
  "thermal_warning": false,
  "timestamp": 1234567890.123
}
```

.

alert

System alerts and warnings.

Payload

```
{
  "level": "warning", // info, warning, error
  "message": "CPU temperature high",
  "timestamp": 1234567890.123
}
```

.

mode_changed

Mode change notification.

Payload

```
{
  "mode": "auto",
  "previous_mode": "manual",
  "timestamp": 1234567890.123
}
```

.

color_changed

Color change broadcast.

Payload

```
{
  "color": "blue",
  "display_name": "Blue 🟦",
  "timestamp": 1234567890.123
}
```

Hardware Integration

Raspberry Pi 5

Specifications - CPU: 2.4GHz quad-core ARM Cortex-A76 - RAM: 4GB or 8GB LPDDR4X - GPIO: 40-pin header, BCM numbering - Camera: 2-lane MIPI CSI connector - Power: 5V/5A USB-C (27W recommended)

GPIO Pin Numbering Uses BCM (Broadcom) numbering, not physical pin numbers.

Camera Module

Supported Cameras - Raspberry Pi Camera Module v2 (8MP) - Raspberry Pi Camera Module v3 (12MP) - Other Picamera2-compatible cameras

Connection - Connect to CSI port (ribbon cable) - Enable via `raspi-config` - Test with `libcamera-hello`

L298N Motor Driver

Specifications - Logic Voltage: 5V - Motor Voltage: 5-35V (typically 12V) - Max Current: 2A per channel - Dual H-bridge for 2 motors

Pin Connections

L298N Pin	Raspberry Pi	Purpose
IN1 (Left)	GPIO 24	Motor direction
IN2 (Left)	GPIO 25	Motor direction
EN (Left)	GPIO 23	Speed (PWM)
IN1 (Right)	GPIO 27	Motor direction
IN2 (Right)	GPIO 22	Motor direction
EN (Right)	GPIO 17	Speed (PWM)
12V	Power supply	Motor power
5V	Raspberry Pi	Logic power
GND	Common ground	Ground

Motor Connections - OUT1, OUT2: Left motor - OUT3, OUT4: Right motor

Power Supply

Recommended Setup - Raspberry Pi: 5V/5A USB-C power supply - Motors: 12V battery or power supply (2A minimum) - Common ground between Pi and motor power

Warning: Do not power Raspberry Pi from L298N 5V output under heavy load.

Development Guide

Adding New Features

Adding a New Tracking Color

1. Edit config.py:

```
AVAILABLE_COLORS['pink'] = {  
    'hsv_min': [140, 100, 100],  
    'hsv_max': [170, 255, 255],  
    'display_name': 'Pink',  
    'emoji': '💖'  
}
```
2. No code changes needed - system will automatically detect new color.

Tuning PID Controllers

1. Adjust in config.py:

```
## Increase responsiveness  
PID_CENTER_KP = 1.0 # More aggressive centering  
  
## Reduce oscillation  
PID_CENTER_KD = 0.2 # More damping
```
2. Test in autonomous mode
3. Iterate until desired behavior achieved

Adding New API Endpoints

1. Edit modules/server.py:

```
@app.get("/api/my-endpoint")  
async def my_endpoint():  
    return {"data": "value"}
```
2. Restart backend
3. Access at `http://<pi-ip>:8000/api/my-endpoint`

Debugging

Enable Debug Logging

Edit config.py:

```
LOG_LEVEL = "DEBUG"
```

View Real-time Logs

```
tail -f robot.log
```

Test Individual Modules

Camera Test

```
from modules.camera import CameraCapture  
camera = CameraCapture(640, 480, 30)  
camera.start()  
frame = camera.get_frame()  
print(f"Frame shape: {frame.shape if frame is not None else 'None'}")  
camera.stop()
```

Motor Test

```
from modules.motors import MotorController
motors = MotorController()
motors.set_speed(30, 30) # Slow forward
time.sleep(2)
motors.stop()
```

Tracking Test

```
from modules.tracking import ColorTracker
import cv2

tracker = ColorTracker('orange')
frame = cv2.imread('test_image.jpg')
result = tracker.detect(frame)
print(result)
```

Performance Optimization

Reduce CPU Usage

```
## Lower camera resolution
CAMERA_WIDTH = 480
CAMERA_HEIGHT = 320
```

Reduce streaming FPS

```
STREAM_FPS = 10
```

Lower JPEG quality

```
JPEG_QUALITY = 50
```

Reduce Network Bandwidth

Smaller stream resolution

```
STREAM_WIDTH = 320
STREAM_HEIGHT = 240
```

Lower quality

```
JPEG_QUALITY = 40
```

Testing Without Hardware

The backend automatically detects missing hardware: - Camera: Uses dummy frames - Motors: Simulates motor control - GPIO: Skips GPIO operations

This allows full development and testing on any machine.

Common Issues

Camera Not Found

Check camera detection

```
libcamera-hello
```

Enable camera interface

```
sudo raspi-config
```

Restart

```
sudo reboot
```

GPIO Permission Denied

Add user to gpio group

```
sudo usermod -a -G gpio $USER
```

Re-Login or reboot

High CPU Temperature

- Lower camera FPS
- Reduce processing resolution
- Add heatsink/fan
- Check THERMAL_LIMIT in config

Motors Not Responding

- Check wiring
- Verify GPIO pins in config
- Test with `cli_control/test_motors.py`
- Check motor power supply

.

License

This project is licensed under the terms specified in the LICENSE file.

.

Support

For issues and questions: - GitHub Issues: <https://github.com/aka-nahal/voyagerbot/issues> - Documentation: See FEATURES_DOCUMENTATION.md

.

Last Updated: November 2025

Version: 1.0

Platform: Raspberry Pi 5 + Python 3.9+

Part 3: Frontend Documentation

Introduction

Table of Contents

1. Overview
2. Technology Stack
3. Architecture
4. Installation & Setup
5. Project Structure
6. Pages & Routing
7. Components
8. State Management
9. AI Integration
10. Mobile App

11. Styling & Theme

12. Development Guide

Overview

VoyagerBot frontend is a modern, responsive web application built with Next.js 14, providing an intuitive interface for controlling and monitoring the robot. It features real-time video streaming, voice-controlled AI commands, comprehensive telemetry visualization, and can be deployed as both a web app and Android mobile app.

Key Features

- **Real-time Control:** WebSocket-based robot control with minimal latency
- **Live Video Streaming:** JPEG video stream with tracking overlay
- **AI Voice Assistant:** Google Gemini-powered voice commands
- **Mobile Support:** Responsive design + native Android app (Capacitor)
- **Telemetry Dashboard:** Real-time system metrics visualization
- **Dark Theme:** Modern dark UI with orange accent colors
- **Offline-Ready:** Works without backend for UI development

Technology Stack

Core Framework

Technology	Version	Purpose
Next.js	14.2.25	React framework with App Router
React	18.3.1	UI library
TypeScript	5.x	Type-safe JavaScript
Node.js	18+	Runtime environment

UI & Styling

Library	Version	Purpose
Tailwind CSS	3.4.17	Utility-first CSS framework
Shadcn UI	Latest	Component library
Radix UI	Various	Headless UI primitives
Lucide React	0.454.0	Icon library
Geist Font	1.3.1	Typography
class-variance-authority	0.7.1	Component variants
clsx	2.1.1	Conditional className utility
tailwind-merge	2.5.5	Tailwind class merging

Real-time Communication

Library	Version	Purpose
Socket.IO Client	4.8.1	WebSocket communication

AI Integration

Library	Version	Purpose
---------	---------	---------

Library	Version	Purpose
@google/generative-ai	0.24.1	Google Gemini AI SDK
@google/genai	1.29.0	Additional Gemini features

Mobile App

Library	Version	Purpose
@capacitor/core	7.4.4	Native app runtime
@capacitor/android	7.4.4	Android platform support
@capacitor/cli	7.4.4	Build tools

Forms & Validation

Library	Version	Purpose
react-hook-form	7.54.1	Form management
zod	3.24.1	Schema validation
@hookform/resolvers	3.9.1	Form validators

Charts & Visualization

Library	Version	Purpose
recharts	2.15.0	Data visualization

UI Components (Radix UI)

Full list of Radix UI components: - Accordion, Alert Dialog, Aspect Ratio, Avatar - Checkbox, Collapsible, Context Menu, Dialog - Dropdown Menu, Hover Card, Label, Menubar - Navigation Menu, Popover, Progress, Radio Group - Scroll Area, Select, Separator, Slider - Slot, Switch, Tabs, Toast, Toggle - Toggle Group, Tooltip

Additional Libraries

Library	Purpose
next-themes	Theme switching
sonner	Toast notifications
cmdk	Command palette
date-fns	Date utilities
embla-carousel-react	Carousel component
react-day-picker	Date picker
react-resizable-panels	Resizable layouts
vaul	Drawer component
input-otp	OTP input

Development Tools

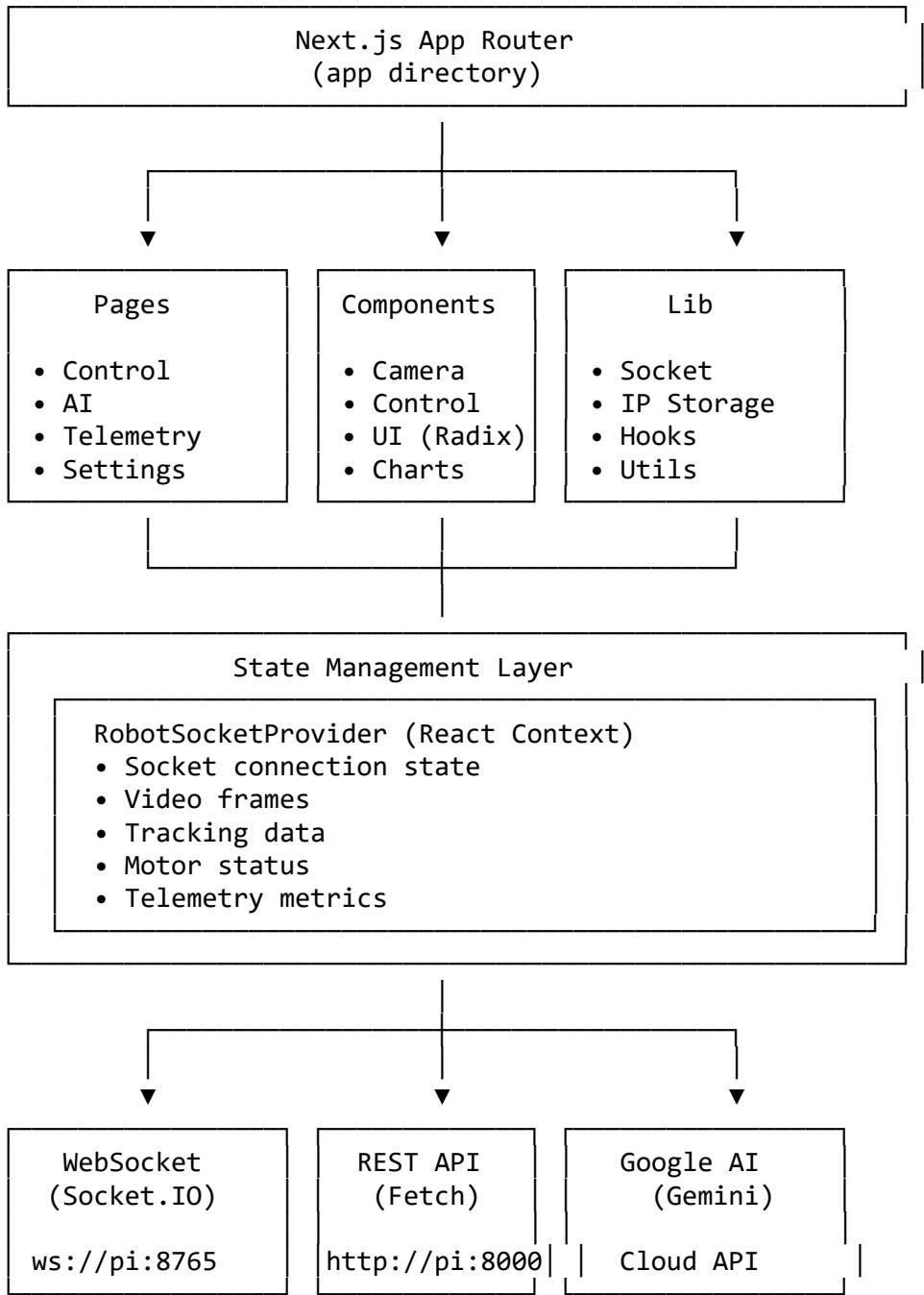
Tool	Version	Purpose
TypeScript	5.x	Static typing
PostCSS	8.5	CSS processing
Autoprefixer	10.4.20	CSS vendor prefixes

Complete Dependencies

See frontend/package.json for the full dependency list.

Architecture

Application Architecture



Data Flow

Backend WebSocket → Socket.IO Client → React Context → Components
User Input → Components → Socket.IO Client → Backend
AI Voice → Google Gemini → Command Parser → Socket.IO → Backend

Page Routing (Next.js App Router)

app/	
├── page.tsx	# Dashboard (/)
├── robot-control/	
│ └── page.tsx	# Robot Control (/robot-control)
├── ai/	
│ └── page.tsx	# AI Assistant (/ai)
├── telemetry/	
│ └── page.tsx	# Telemetry (/telemetry)
├── settings/	
│ └── page.tsx	# Settings (/settings)
├── command-center/	
│ └── page.tsx	# Command Center (/command-center)
├── operations/	
│ └── page.tsx	# Operations (/operations)
├── systems/	
│ └── page.tsx	# Systems (/systems)
├── intelligence/	
│ └── page.tsx	# Intelligence (/intelligence)
└── agent-network/	
└── page.tsx	# Agent Network (/agent-network)

Installation & Setup

Prerequisites

- **Node.js:** 18.x or higher
- **npm, yarn, or pnpm:** Package manager
- **Backend:** VoyagerBot backend running (for full functionality)

Installation Steps

1. Clone Repository

```
git clone https://github.com/aka-nahal/voyagerbot.git
cd voyagerbot/frontend
```

2. Install Dependencies

```
npm install
# or
yarn install
# or
pnpm install
```

3. Configure Environment (Optional) Create .env.local for custom configuration:

```
NEXT_PUBLIC_API_URL=http://192.168.1.100:8000
NEXT_PUBLIC_WS_URL=ws://192.168.1.100:8765
```

4. Run Development Server

```
npm run dev
# or
yarn dev
```


hooks/	# Custom React hooks
└─ useLiveAssistant.ts	# AI voice assistant hook
└─ ...	
styles/	# Global styles
└─ globals.css	# Tailwind imports & customs
public/	# Static assets
└─ favicon.ico	
└─ ...	
android/	# Capacitor Android project
└─ app/	
└─ build.gradle	
capacitor.config.ts	# Capacitor configuration
next.config.mjs	# Next.js configuration
tailwind.config.ts	# Tailwind CSS config
tsconfig.json	# TypeScript config
package.json	# Dependencies & scripts
components.json	# Shadcn UI config

Pages & Routing

Main Dashboard (app/page.tsx)

Route: /

Purpose: Central hub with navigation to all sections.

Features: - Section navigation tabs - IP address configuration dialog - Connection status indicator - Responsive layout (mobile/desktop) - Animated preloader

Sections: - CONTROL: Robot control interface - AI: Voice assistant - TELEMETRY: System monitoring - SETTINGS: Configuration

Key Components:

- IPAddressDialog: Initial IP setup
- Navigation tabs
- Sidebar (desktop)
- Mobile menu

Robot Control Page (app/robot-control/page.tsx)

Route: /robot-control

Purpose: Main robot control interface.

Features: - Live camera feed with overlay - Manual control pad (joystick) - Mode switching (Manual/Auto/Patrol) - Color selection for tracking - Emergency stop button - Status display

Components Used:

- CameraFeed: Video stream display
- ControlPad: Joystick controller
- Card: UI containers
- Button: Control buttons
- Badge: Status indicators

State:

```
const {
  isConnected,
  videoFrame,
  trackingData,
  motorStatus,
  sendManualControl,
  changeMode,
  emergencyStop,
  changeColor
} = useRobotSocket();
```

.

AI Page (app/ai/page.tsx)

Route: /ai

Purpose: AI-powered voice control interface.

Features: - Voice assistant (Google Gemini) - Voice input via microphone - Text input alternative - Command queue visualization - Integrated camera feed - Emergency stop

AI Capabilities: - Natural language understanding - Robot control commands - Context-aware responses - Voice synthesis

Hook Used:

```
const {
  status,
  isListening,
  transcript,
  response,
  commandQueue,
  startListening,
  stopListening,
  sendTextMessage,
  cancelCommand
} = useLiveAssistant(ipAddress);
```

.

Telemetry Page (app/telemetry/page.tsx)

Route: /telemetry

Purpose: System monitoring and metrics visualization.

Features: - Real-time metrics display - CPU/RAM usage charts - Temperature monitoring - FPS counters - Connection status - Motor status - Color-coded warnings

Metrics Displayed: - CPU usage (%) - CPU temperature (°C) - RAM usage (MB, %) - Processing FPS - Streaming FPS - Motor speeds - Connection status

Components:

- Card: Metric containers
- Progress: CPU/RAM bars
- Badge: Status indicators
- Charts: Usage graphs (if implemented)

.

Settings Page (app/settings/page.tsx)

Route: /settings

Purpose: Application configuration.

Features: - Backend IP address management - Connection testing - Network information - Control preferences - System settings

Functions:

- Change backend IP
- Test connection
- View network details
- Clear saved settings

.

Components

CameraFeed (components/camera-feed.tsx)

Purpose: Display live video stream with tracking overlay.

Props:

```
interface CameraFeedProps {  
  videoFrame?: string;           // Base64 JPEG  
  trackingData?: TrackingData;  
  isConnected: boolean;  
  className?: string;  
}
```

Features: - Base64 JPEG rendering - Detection marker overlay - 3x3 grid overlay - FPS counter - Connection indicator - Tracking info display - Distance indicator

Tracking Overlay: - Green circle: Marker detected - Crosshair: Marker center - Distance text - Confidence indicator

.

ControlPad (components/control-pad.tsx)

Purpose: Joystick-based robot control.

Props:

```
interface ControlPadProps {
  onCommand: (direction: Direction, speed: number) => void;
  disabled?: boolean;
}
```

Features: - Touch/mouse joystick - 8-directional control - Speed adjustment (30-100%) - Rotation buttons - Command repeat - Visual feedback

Controls: - Joystick: Forward, backward, left, right, diagonals - Left/Right buttons: Rotation - +/- buttons: Speed adjustment - Current command display

Usage:

```
<ControlPad
  onCommand={{(direction, speed) => {
    sendManualControl(direction, speed);
  }}}
  disabled={!isConnected}
/>
```

.

Shadcn UI Components (components/ui/)

Pre-built, customizable components from Shadcn UI:

Layout: - Card: Container with header/footer - Separator: Divider line - ScrollArea: Scrollable container - Tabs: Tab navigation

Inputs: - Button: Action button - Input: Text input - Select: Dropdown select - Checkbox: Checkbox input - Switch: Toggle switch - Slider: Range slider - RadioGroup: Radio buttons

Feedback: - Toast: Notification popup - Dialog: Modal dialog - AlertDialog: Confirmation dialog - Progress: Progress bar - Badge: Status badge - Tooltip: Hover tooltip

Navigation: - NavigationMenu: Nav menu - Menubar: Menu bar - DropdownMenu: Dropdown - ContextMenu: Right-click menu

Overlays: - Popover: Popup content - HoverCard: Hover card - Sheet: Side panel

.

State Management

RobotSocketProvider (lib/robot-socket.tsx)

Purpose: Centralized Socket.IO connection and state management.

Context Provider:

```
<RobotSocketProvider>
  {/* App components */}
</RobotSocketProvider>
```

Hook:

```
const {
  // Connection state
  isConnected,
  ipAddress,
```



```

setIpAddress,

// Data streams
videoFrame,          // Base64 JPEG
trackingData,        // Marker detection
motorStatus,         // Motor state
telemetry,           // System metrics
alerts,              // System alerts

// Control functions
sendManualControl,   // Send movement commands
changeMode,          // Switch manual/auto
sendAutoCommand,     // Auto mode commands
emergencyStop,       // Emergency stop
changeColor,         // Change tracking color

// Configuration
fetchColors,         // Get available colors
availableColors      // Color list
} = useRobotSocket();

```

State Structure:

```

interface RobotSocketState {
  isConnected: boolean;
  ipAddress: string;
  videoFrame: string | null;
  trackingData: TrackingData | null;
  motorStatus: MotorStatus | null;
  telemetry: Telemetry | null;
  alerts: Alert[];
  availableColors: Color[];
}

```

WebSocket Events Handled: - connect / disconnect: Connection state - video_frame: Frame updates - tracking_data: Tracking updates - motor_status: Motor updates - telemetry: Metrics updates - alert: Alert messages - color_changed: Color updates

IP Address Storage (lib/ip-storage.ts)

Purpose: Persist backend IP address in localStorage.

Functions:

```

// Get stored IP
const ip = getStoredIpAddress(): string | null

// Save IP
setStoredIpAddress(ip: string): void

// Validate IP format
const isValid = validateIpAddress(ip: string): boolean

// Format IP (remove whitespace, etc.)
const formatted = formatIpAddress(ip: string): string

```

```
// Get full backend URL
const url = getBackendUrl(ip: string): string
```

Usage:

```
import { getStoredIpAddress, setStoredIpAddress } from '@lib/ip-storage';
```

```
// On app Load
const savedIp = getStoredIpAddress();
```

```
// On IP change
setStoredIpAddress('192.168.1.100');
```

.

AI Integration

Google Gemini Live API

Purpose: Natural language voice control for robot.

Hook: useLiveAssistant (hooks/useLiveAssistant.ts)

Setup:

```
const {
  status,           // 'ready' | 'listening' | 'processing'
  isListening,
  transcript,       // Current speech transcription
  response,         // AI response text
  commandQueue,    // Pending commands
  startListening,
  stopListening,
  sendTextMessage,
  cancelCommand
} = useLiveAssistant(ipAddress);
```

Features: - Speech-to-text (Web Speech API) - Google Gemini Live API integration - Command parsing and execution - Text-to-speech responses - Command queue management - Context awareness

Command Examples:

User: "Move forward"

AI: Executes forward movement

User: "Follow the blue marker"

AI: Changes color to blue, enables auto mode

User: "Stop"

AI: Emergency stop

User: "What's the CPU temperature?"

AI: Queries telemetry, responds with temperature

Configuration:

```
// Gemini API Key (set in environment)
NEXT_PUBLIC_GEMINI_API_KEY=your_api_key_here
```

System Prompt: The AI is configured with robot control context: - Available commands - Current status - Safety protocols - Response formatting

Mobile App

Capacitor Android App

Purpose: Native Android app version of the frontend.

Build Configuration: capacitor.config.ts

```
const config: CapacitorConfig = {
  appId: 'com.voyagerbot.app',
  appName: 'VoyagerBot',
  webDir: 'out',
  server: {
    androidScheme: 'https'
  }
};
```

Building Android App

1. Build Next.js

```
npm run build:android
# or manually:
npm run build
npx cap sync android
```

2. Open in Android Studio

```
npm run android:open
# or manually:
npx cap open android
```

3. Build APK

- In Android Studio: Build → Build Bundle(s) / APK(s) → Build APK(s)
- Or use PowerShell scripts:

```
.\build-apk.ps1
.\rebuild-apk.ps1
```

Android Features

- Native app performance
- Offline capabilities
- System integration
- Full-screen mode
- Camera access (if needed)

Android Requirements

- Android Studio

- Java JDK 11+
- Android SDK
- Gradle

Styling & Theme

Tailwind CSS Configuration

File: tailwind.config.ts

Key Settings:

```
{
  darkMode: ['class'],
  content: [
    './pages/**/*..{js,ts,jsx,tsx,mdx}',
    './components/**/*..{js,ts,jsx,tsx,mdx}',
    './app/**/*..{js,ts,jsx,tsx,mdx}'
  ],
  theme: {
    extend: {
      colors: { /* HSL color variables */ },
      borderRadius: { /* Custom radii */ },
      keyframes: { /* Animations */ }
    }
  }
}
```

Color System

Uses CSS custom properties (HSL):

```
:root {
  --background: 0 0% 100%;
  --foreground: 0 0% 3.9%;
  --primary: 25 95% 53%; /* Orange */
  --secondary: 0 0% 96.1%;
  /* ... */
}

.dark {
  --background: 0 0% 3.9%; /* Dark gray */
  --foreground: 0 0% 98%;
  --primary: 25 95% 53%; /* Orange accent */
  /* ... */
}
```

Primary Colors: - Background: Neutral 900 (dark) - Foreground: Neutral 50 (light) -
Primary: Orange (robot theme) - Secondary: Neutral 800

Typography

Font: Geist (sans-serif)

```
import { GeistSans } from 'geist/font/sans';
```

```
<html className={GeistSans.className}>
```

Global Styles

File: styles/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Custom CSS variables */
@layer base {
  :root {
    /* Color variables */
  }
}
```

Component Styling

Uses utility-first Tailwind classes:

```
<div className="flex flex-col gap-4 p-6 bg-neutral-900 rounded-lg">
  <h1 className="text-2xl font-bold text-orange-500">Title</h1>
  <p className="text-neutral-300">Description</p>
</div>
```

Responsive Design

Tailwind breakpoints: - sm: 640px - md: 768px - lg: 1024px - xl: 1280px - 2xl: 1536px

Example:

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {/* Responsive grid */}
</div>
```

.

Development Guide

Adding New Pages

1. **Create page file:**

```
mkdir app/my-page
touch app/my-page/page.tsx
```

2. **Implement page:**

```
'use client';

export default function MyPage() {
  return (
    <div className="container mx-auto p-6">
      <h1 className="text-3xl font-bold">My Page</h1>
    </div>
```

```
    );  
  }  
}
```

3. **Add to navigation:** Update `app/page.tsx` navigation.

Adding New Components

1. **Create component:**

```
touch components/my-component.tsx
```

2. **Implement component:**

```
interface MyComponentProps {  
  title: string;  
}  
  
export function MyComponent({ title }: MyComponentProps) {  
  return <div>{title}</div>;  
}
```

3. **Use component:**

```
import { MyComponent } from '@components/my-component';  
  
<MyComponent title="Hello" />
```

Adding Shadcn UI Components

```
npx shadcn-ui@latest add button  
npx shadcn-ui@latest add card  
npx shadcn-ui@latest add dialog
```

Custom Hooks

Create in `hooks/` directory:

```
// hooks/useMyHook.ts  
export function useMyHook() {  
  const [state, setState] = useState();  
  
  return { state, setState };  
}
```

Connecting to Backend

Use `useRobotSocket` hook:

```
'use client';  
  
import { useRobotSocket } from '@lib/robot-socket';  
  
export default function MyPage() {  
  const { isConnected, videoFrame, sendManualControl } = useRobotSocket();  
  
  return (  
    <div>  
      {isConnected ? 'Connected' : 'Disconnected'}  
    </div>  
  )  
}
```

```
);  
}
```

Environment Variables

Create `.env.local`:

```
NEXT_PUBLIC_GEMINI_API_KEY=your_key_here  
NEXT_PUBLIC_DEFAULT_BACKEND_IP=192.168.1.100
```

Access in code:

```
const apiKey = process.env.NEXT_PUBLIC_GEMINI_API_KEY;
```

Testing

Development server:

```
npm run dev
```

Production build:

```
npm run build  
npm start
```

Linting:

```
npm run lint
```

Debugging

React DevTools: Install browser extension

Console Logging:

```
console.log('Debug:', { isConnected, videoFrame });
```

Network Tab: Monitor WebSocket/HTTP requests

Next.js DevTools: Built-in developer overlay

Performance Optimization

Image Optimization:

```
import Image from 'next/image';
```

```
<Image src="/image.jpg" width={500} height={500} alt="Image" />
```

Code Splitting:

```
import dynamic from 'next/dynamic';
```

```
const HeavyComponent = dynamic(() => import('./HeavyComponent'), {  
  loading: () => <p>Loading...</p>  
});
```

Memoization:

```
import { memo, useMemo, useCallback } from 'react';
```

```
const MemoizedComponent = memo(MyComponent);

const memoizedValue = useMemo(() => computeValue(), [deps]);
const memoizedCallback = useCallback(() => {}, [deps]);
```

Common Issues

Module not found:

```
npm install
## or
rm -rf node_modules package-lock.json
npm install
```

Type errors:

```
npm run build
## Check TypeScript errors
```

Port in use:

```
## Change port
npm run dev -- -p 3001
```

WebSocket connection failed: - Check backend IP address - Verify backend is running - Check firewall settings - Ensure same network

.

Deployment

Web Deployment

Vercel (recommended):

```
npm install -g vercel
vercel
```

Other platforms: - Netlify - AWS Amplify - Docker

Build:

```
npm run build
npm start
```

Android Deployment

Debug APK:

```
npm run build:android
## Open Android Studio
## Build → Build APK
```

Release APK: 1. Configure signing in Android Studio 2. Build → Generate Signed Bundle/APK 3. Follow wizard

.

API Reference

useRobotSocket Hook

```
const {
  // Connection
  isConnected: boolean,
  ipAddress: string,
  setIpAddress: (ip: string) => void,

  // Data
  videoFrame: string | null,
  trackingData: TrackingData | null,
  motorStatus: MotorStatus | null,
  telemetry: Telemetry | null,
  alerts: Alert[],
  availableColors: Color[],

  // Actions
  sendManualControl: (direction: Direction, speed: number) => void,
  changeMode: (mode: 'manual' | 'auto') => void,
  sendAutoCommand: (command: string) => void,
  emergencyStop: () => void,
  changeColor: (color: string) => void,
  fetchColors: () => Promise<void>
} = useRobotSocket();
```

useLiveAssistant Hook

```
const {
  status: 'ready' | 'listening' | 'processing',
  isListening: boolean,
  transcript: string,
  response: string,
  commandQueue: Command[],
  startListening: () => void,
  stopListening: () => void,
  sendTextMessage: (text: string) => void,
  cancelCommand: (id: string) => void
} = useLiveAssistant(ipAddress: string);
```

.

License

This project is licensed under the terms specified in the LICENSE file.

.

Support

For issues and questions: - GitHub Issues: <https://github.com/aka-nahal/voyagerbot/issues> - Documentation: See BACKEND_DOCUMENTATION.md and FEATURES_DOCUMENTATION.md

.

Part 4: Features Reference

Introduction

Executive Summary

VoyagerBot is an advanced autonomous robot control system built on Raspberry Pi 5, featuring real-time video streaming, color-based object tracking, autonomous navigation, and AI-powered voice control. The system consists of a Python-based backend running on Raspberry Pi hardware and a modern Next.js frontend application accessible via web browsers and mobile devices.

Table of Contents

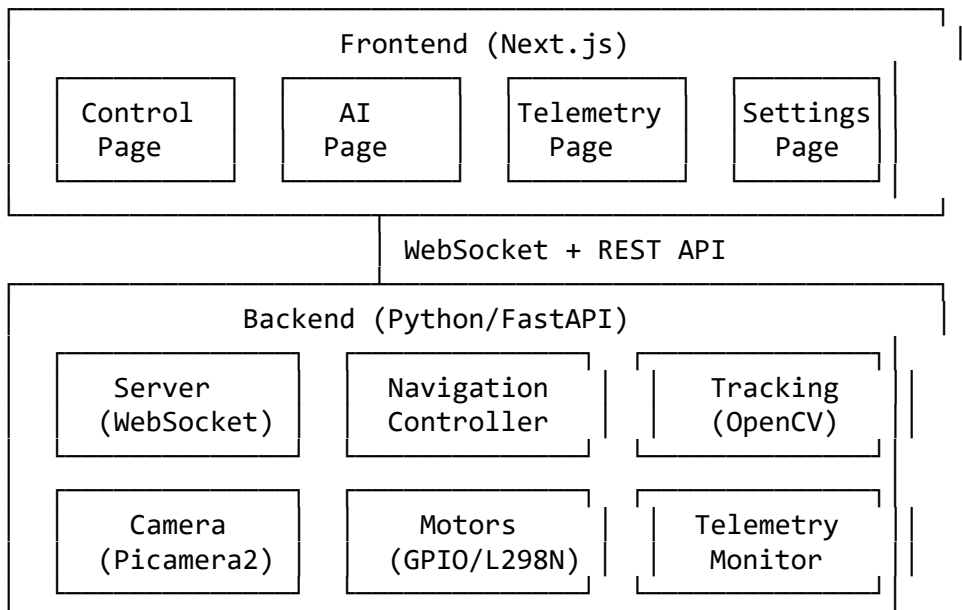
- 1. [System Architecture](#)
- 2. [Backend Features](#)
- 3. [Frontend Features](#)
- 4. [Communication Protocol](#)
- 5. [Hardware Integration](#)
- 6. [Technical Specifications](#)

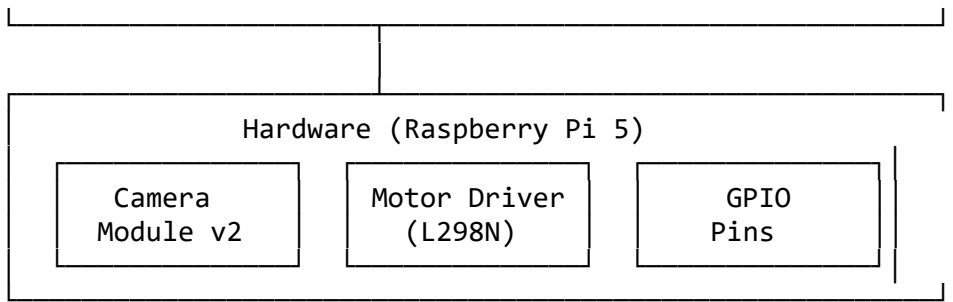
System Architecture

Overview

VoyagerBot employs a modular, multi-threaded architecture with clear separation between hardware control, computer vision, navigation, and communication layers.

Component Diagram





Backend Features

1. Camera Module (`modules/camera.py`)

Features

- **High-Performance Capture:** Utilizes Picamera2 library for efficient video capture
- **Configurable Resolution:** Supports 640x480 capture resolution at 30 FPS
- **Thread-Safe Frame Access:** Implements locking mechanism for concurrent access
- **Automatic Reconnection:** Self-healing camera connection with error recovery
- **FPS Monitoring:** Real-time frame rate calculation and reporting

Technical Details

- **Capture Format:** RGB888
- **Threading Model:** Dedicated capture thread with frame buffering
- **Error Handling:** Graceful degradation with automatic reconnection attempts
- **Performance:** Maintains consistent frame rate with minimal CPU overhead

Key Methods

- `start()`: Initializes camera and starts capture thread
- `get_frame()`: Thread-safe frame retrieval
- `get_fps()`: Returns current capture FPS
- `stop()`: Clean shutdown with resource cleanup

2. Color Tracking Module (`modules/tracking.py`)

Features

- **Multi-Color Support:** Tracks 8 different colors (Orange, Red, Blue, Green, Yellow, Purple, Black, Brown)
- **Dynamic Color Switching:** Runtime color change without system restart
- **HSV-Based Detection:** Robust color detection using HSV color space
- **Distance Estimation:** Calculates distance to tracked object using blob area
- **Confidence Scoring:** Provides detection confidence based on blob characteristics
- **Morphological Filtering:** Noise reduction using OpenCV morphological operations

Color Configuration

Each color includes: - HSV min/max ranges for detection - Display name and emoji for UI - Calibrated detection parameters

Detection Algorithm

1. **Color Space Conversion:** BGR → HSV
2. **Color Thresholding:** In-range filtering
3. **Morphological Operations:** Noise removal (open + close)
4. **Contour Detection:** Find valid blobs
5. **Area Filtering:** Filter by min/max blob area
6. **Centroid Calculation:** Calculate marker center using moments
7. **Distance Estimation:** Inverse square relationship from blob area

Key Methods

- `detect(frame)`: Main detection function returning tracking data
- `set_color(color_name)`: Dynamically change tracking color
- `_estimate_distance(blob_area)`: Calculate distance in centimeters
- `_calculate_confidence()`: Compute detection confidence score

3. Motor Control Module (`modules/motors.py`)

Features

- **Differential Drive System:** Independent left/right motor control
- **PWM Speed Control:** Precise speed control using PWM signals
- **Direction Control:** Forward/backward movement via H-bridge
- **Speed Ramping:** Gradual acceleration/deceleration for smooth operation
- **Watchdog Timer:** Automatic motor stop on communication timeout
- **Simulation Mode:** Works without hardware for testing

Hardware Interface

- **Motor Driver:** L298N H-bridge
- **GPIO Pins:** BCM pin numbering
 - Left Motor: EN=23, IN1=24, IN2=25
 - Right Motor: EN=17, IN1=27, IN2=22
- **PWM Frequency:** 1000 Hz

Safety Features

- **Watchdog Timeout:** 0.5 seconds default timeout
- **Speed Limits:** Configurable max speeds (100% manual, 80% autonomous)
- **Emergency Stop:** Immediate motor halt capability
- **Automatic Cleanup:** GPIO cleanup on shutdown

Key Methods

- `set_speed(left, right)`: Set motor speeds with ramping
- `stop()`: Emergency stop all motors
- `get_status()`: Return current motor state
- `_ramp_speeds()`: Gradual speed transitions

4. Navigation Controller (modules/navigation.py)

Features

- **State Machine:** Robust state management (IDLE, SEARCHING, LOCKED, FOLLOWING, LOST, STOPPED)
- **PID Controllers:** Dual PID system for centering and distance control
- **Autonomous Following:** Intelligent object following with dynamic speed adjustment
- **Sideways Movement Detection:** Tracks horizontal velocity for predictive steering
- **Zone-Based Control:** FRIDAY-inspired zone-based navigation (left/center/right)
- **Adaptive Speed:** Speed adjustment based on distance and movement patterns

State Machine

1. **IDLE:** System ready, waiting for mode change
2. **SEARCHING:** Rotating to find marker
3. **LOCKED:** Marker detected, preparing to follow
4. **FOLLOWING:** Actively tracking and following marker
5. **LOST:** Marker lost, waiting for reacquisition
6. **STOPPED:** Emergency stop state

Navigation Algorithm

- **Zone Detection:** Divides frame into left (0-40%), center (40-60%), right (60-100%)
- **Distance-Based Speed:** Closer objects = slower speed, farther = faster
- **Dynamic Steering:** Adjusts turn angle based on horizontal velocity
- **Sudden Movement Detection:** Accelerates response for fast-moving objects

PID Tuning

- **Centering PID:** $K_p=0.8$, $K_i=0.0$, $K_d=0.1$
- **Distance PID:** $K_p=0.5$, $K_i=0.0$, $K_d=0.05$
- **Anti-Windup:** Integral term clamping to prevent overshoot

Key Methods

- `update(tracking_data)`: Main navigation update function
- `set_mode(mode)`: Switch between manual/auto modes
- `emergency_stop()`: Trigger emergency stop
- `_following_behavior()`: Generate motor commands for following

5. Telemetry Monitor (modules/telemetry.py)

Features

- **System Metrics:** CPU usage, temperature, RAM usage
- **Performance Tracking:** Processing and streaming FPS monitoring
- **Network Statistics:** Bytes sent/received tracking
- **Thermal Monitoring:** CPU temperature warnings
- **Uptime Tracking:** System uptime calculation

Metrics Collected

- CPU temperature (°C)

- CPU usage (%)
- RAM usage (MB, %)
- Processing FPS
- Streaming FPS
- Network I/O (bytes)
- Uptime (seconds)
- Thermal warnings

Key Methods

- `collect_metrics()`: Gather all system metrics
- `get_cpu_temperature()`: Read CPU temp from thermal zone
- `record_processing_frame()`: Track processing FPS
- `record_streaming_frame()`: Track streaming FPS

6. Communication Server (`modules/server.py`)

Features

- **WebSocket Server**: Real-time bidirectional communication using Socket.IO
- **REST API**: HTTP endpoints for configuration and status
- **CORS Support**: Cross-origin resource sharing enabled
- **Client Management**: Tracks connected clients
- **Message Queue**: Async message processing for thread-safe operations
- **Event Handlers**: Comprehensive event handling for all robot commands

REST API Endpoints

`GET /api/status`

Returns current system status including: - Navigation mode and state - Tracking color information - Connected client count - Motor status - Telemetry data

`GET /api/config`

Returns all configuration parameters: - Camera settings (width, height, FPS) - Tracking parameters (HSV ranges, blob areas) - Motor limits - PID controller gains

`GET /api/health`

Health check endpoint returning: - System status - Uptime - Timestamp

`GET /api/colors`

Returns available tracking colors with: - Color keys and display names - HSV ranges - Current active color

`POST /api/config`

Update configuration parameters (requires proper implementation)

`POST /api/calibrate/distance`

Calibrate distance estimation using known measurements

WebSocket Events

Client → Server

- `manual_control`: Send manual movement commands
- `mode_change`: Switch between manual/auto modes
- `auto_command`: Autonomous mode commands
- `emergency_stop`: Trigger emergency stop
- `change_color`: Change tracking color
- `settings_update`: Update system settings

Server → Client

- `connected`: Connection confirmation
- `video_frame`: JPEG-encoded video frames (base64)
- `tracking_data`: Marker detection data
- `motor_status`: Current motor speeds and state
- `telemetry`: System telemetry metrics
- `alert`: System alerts and warnings
- `mode_changed`: Mode change notification
- `color_changed`: Color change broadcast

Key Methods

- `send_video_frame()`: Broadcast video frames to all clients
- `send_tracking_data()`: Send tracking information
- `send_motor_status()`: Broadcast motor state
- `send_telemetry()`: Send system metrics
- `send_alert()`: Send alert messages

7. Main Controller (`main.py`)

Features

- **Orchestration**: Coordinates all system modules
- **Thread Management**: Manages processing, streaming, and telemetry threads
- **Signal Handling**: Graceful shutdown on SIGINT/SIGTERM
- **Error Recovery**: Comprehensive error handling and logging
- **Queue Management**: Frame queues for processing and streaming pipelines

Thread Architecture

1. **Processing Thread**: Camera capture → Tracking → Navigation → Motor control
2. **Streaming Thread**: Frame encoding → WebSocket broadcast
3. **Telemetry Thread**: Metrics collection → WebSocket broadcast

Key Methods

- `start()`: Initialize and start all components
- `_processing_loop()`: Main processing pipeline
- `_streaming_loop()`: Video streaming pipeline
- `_telemetry_loop()`: Telemetry update loop
- `stop()`: Graceful shutdown

Frontend Features

1. Main Dashboard (`app/page.tsx`)

Features

- **Responsive Layout:** Mobile-first design with desktop sidebar
- **Section Navigation:** Tab-based navigation between sections
- **IP Address Management:** Initial IP configuration dialog
- **Connection Status:** Real-time connection indicator
- **Preloader:** Animated loading screen
- **Theme:** Dark theme with orange accent colors

Navigation Sections

- **CONTROL:** Robot control interface
- **AI:** Voice assistant and AI features
- **TELEMETRY:** System metrics and monitoring
- **SETTINGS:** Configuration and preferences

2. Robot Control Page (`app/robot-control/page.tsx`)

Features

- **Live Camera Feed:** Real-time video streaming with overlay
- **Mode Selection:** Manual, Autonomous, Patrol modes
- **Color Selection:** Dynamic tracking color switching
- **Control Pad:** Joystick-based movement control
- **Emergency Stop:** Immediate robot halt button
- **Status Display:** Real-time system status information

Camera Feed Component (`components/camera-feed.tsx`)

- **Video Display:** Base64 JPEG frame rendering
- **Detection Overlay:** Visual marker detection indicators
- **Grid Overlay:** 3x3 grid for spatial reference
- **Connection Status:** Visual connection indicator
- **FPS Counter:** Real-time frame rate display
- **Tracking Info:** Marker detection status and distance

Control Pad Component (`components/control-pad.tsx`)

- **Joystick Control:** Touch/mouse-based directional control
- **Rotation Controls:** Left/right rotation buttons
- **Speed Control:** Adjustable speed (30-100%)
- **Command Repeating:** Configurable repeat interval
- **Visual Feedback:** Real-time command display

Features

- **Touch Support:** Full touch event handling
- **Mouse Support:** Desktop mouse control
- **Command Visualization:** Current command display
- **Speed Adjustment:** +/- buttons for speed control

- **Rotation Tracking:** Angle display for rotations

3. AI Page ([app/ai/page.tsx](#))

Features

- **Voice Assistant:** Google Gemini Live API integration
- **Voice Input:** Microphone-based command input
- **Text Input:** Alternative text-based commands
- **Command Queue:** Visual queue of pending/executing commands
- **Camera Feed:** Integrated camera view
- **Emergency Stop:** Stop all AI operations

Voice Assistant Features ([hooks/useLiveAssistant.ts](#))

- **Speech Recognition:** Browser Web Speech API
- **AI Processing:** Gemini Live API for command interpretation
- **Command Execution:** Robot control command execution
- **Audio Playback:** Text-to-speech responses
- **Status Management:** Ready/Listening/Processing states
- **Queue Management:** Command queuing and cancellation

Command Types Supported

- Movement commands (forward, backward, left, right, stop)
- Mode changes (manual, autonomous)
- Color changes
- Emergency stop
- Custom commands via AI interpretation

4. Telemetry Page ([app/telemetry/page.tsx](#))

Features

- **Real-Time Metrics:** Live system performance data
- **Visual Charts:** CPU and memory history graphs
- **Sensor Display:** Comprehensive sensor readings
- **Status Indicators:** Color-coded status indicators
- **Responsive Grid:** Adaptive layout for different screen sizes

Metrics Displayed

- **CPU Usage:** Percentage with warning thresholds
- **RAM Usage:** MB and percentage
- **Connection Status:** Online/offline indicator
- **FPS:** Processing and streaming frame rates
- **Temperature:** CPU temperature with thermal warnings
- **Motor Status:** Left/right motor speeds

5. Settings Page (app/settings/page.tsx)

Features

- **IP Configuration:** Backend IP address management
- **Connection Testing:** Test connection to robot
- **Network Information:** Display network details
- **Control Preferences:** Manual control settings
- **Theme Settings:** Dark/light theme toggle (if implemented)

6. Socket Communication (lib/robot-socket.tsx)

Features

- **WebSocket Client:** Socket.IO client implementation
- **Auto-Reconnection:** Automatic reconnection on disconnect
- **Event Handling:** Comprehensive event listeners
- **State Management:** React context for global state
- **IP Management:** Dynamic IP address handling

Socket Events Handled

- `connect/disconnect`: Connection state management
- `video_frame`: Frame reception and display
- `tracking_data`: Tracking information updates
- `motor_status`: Motor state updates
- `telemetry`: System metrics updates
- `alert`: Alert message handling
- `color_changed`: Color change notifications

Key Functions

- `sendManualControl()`: Send movement commands
- `changeMode()`: Switch navigation mode
- `sendAutoCommand()`: Autonomous commands
- `emergencyStop()`: Emergency stop
- `changeColor()`: Change tracking color
- `fetchColors()`: Load available colors

7. IP Address Management (lib/ip-storage.ts)

Features

- **Local Storage:** Browser-based IP persistence
- **Validation:** IP address format validation
- **Formatting:** IP address normalization
- **URL Generation:** Backend URL construction

Functions

- `getStoredIpAddress()`: Retrieve saved IP
- `setStoredIpAddress()`: Save IP address
- `validateIpAddress()`: Validate IP format

- `formatIpAddress()`: Normalize IP format
- `getBackendUrl()`: Construct backend URL

8. UI Components

Component Library

Built on **Shadcn UI** and **Radix UI**: - **Dialog**: Modal dialogs (IP address dialog) - **Button**: Styled buttons with variants - **Card**: Content cards with headers - **Input**: Form inputs - **Badge**: Status badges - **Toast**: Notification toasts

Design System

- **Colors**: Dark theme (neutral-900) with orange accents
- **Typography**: Geist font family
- **Spacing**: Consistent spacing scale
- **Responsive**: Mobile-first breakpoints
- **Animations**: Smooth transitions and animations

Communication Protocol

WebSocket Protocol (Socket.IO)

Connection

- **Transport**: WebSocket with polling fallback
- **Reconnection**: Automatic with exponential backoff
- **Timeout**: 30 seconds connection timeout
- **Heartbeat**: Built-in Socket.IO heartbeat

Message Format

Video Frame

```
{
  "type": "video_frame",
  "data": "base64_encoded_jpeg",
  "timestamp": 1234567890.123,
  "frame_number": 12345
}
```

Tracking Data

```
{
  "type": "tracking_data",
  "marker_detected": true,
  "position": { "x": 320, "y": 240 },
  "distance_cm": 45.5,
  "confidence": 0.85,
  "blob_area": 5000,
  "timestamp": 1234567890.123
}
```

Motor Status

```
{
  "type": "motor_status",
```

```
"left_speed": 50,  
"right_speed": 50,  
"mode": "manual",  
"state": "idle",  
"timestamp": 1234567890.123  
}  
  
Telemetry  
{  
  "type": "telemetry",  
  "cpu_temp": 45.2,  
  "cpu_usage": 25.5,  
  "ram_usage_mb": 512.3,  
  "ram_total_mb": 2048.0,  
  "ram_percent": 25.0,  
  "fps_processing": 28.5,  
  "fps_streaming": 14.2,  
  "uptime_seconds": 3600.0,  
  "thermal_warning": false,  
  "network_bytes_sent": 1048576,  
  "network_bytes_recv": 524288,  
  "timestamp": 1234567890.123  
}
```

Hardware Integration

Raspberry Pi 5 Specifications

- **CPU:** 2.4GHz quad-core ARM Cortex-A76
- **RAM:** 4GB/8GB options
- **GPIO:** 40-pin header with BCM numbering
- **Camera:** Raspberry Pi Camera Module v2/v3
- **Power:** 5V USB-C power supply

Motor Driver (L298N)

- **Voltage:** 5V logic, 12V motor supply
- **Current:** Up to 2A per channel
- **Control:** H-bridge for bidirectional control
- **PWM:** Speed control via enable pins

GPIO Pin Mapping

Left Motor:

```
EN   → GPIO 23 (PWM)  
IN1  → GPIO 24 (Direction)  
IN2  → GPIO 25 (Direction)
```

Right Motor:

```
EN   → GPIO 17 (PWM)  
IN1  → GPIO 27 (Direction)  
IN2  → GPIO 22 (Direction)
```

Technical Specifications

Backend Requirements

- **Python:** 3.9+
- **Libraries:**
 - picamera2 >= 0.3.12
 - opencv-python >= 4.8.0
 - numpy >= 1.24.0
 - fastapi >= 0.104.0
 - uvicorn >= 0.24.0
 - python-socketio >= 5.10.0
 - RPi.GPIO >= 0.7.1
 - psutil >= 5.9.0

Frontend Requirements

- **Node.js:** 18+
- **Framework:** Next.js 14 (App Router)
- **UI Libraries:**
 - React 18.3.1
 - Shadcn UI / Radix UI
 - Tailwind CSS
 - Socket.IO Client 4.8.1
 - Google Generative AI SDK

Performance Metrics

- **Camera Capture:** 30 FPS @ 640x480
- **Video Streaming:** 15 FPS @ 480x320
- **Processing Latency:** < 33ms per frame
- **Network Latency:** < 50ms (local network)
- **Motor Response:** < 100ms command execution

Configuration Parameters

Camera

- Resolution: 640x480
- FPS: 30
- Stream Resolution: 480x320
- Stream FPS: 15
- JPEG Quality: 65

Tracking

- Min Blob Area: 500 pixels
- Max Blob Area: 50,000 pixels
- Marker Lost Timeout: 3.0 seconds
- Target Distance: 20cm (minimum safe distance)
- Max Tracking Distance: 500cm

Motors

- Max Speed: 100% (manual), 80% (autonomous)
- PWM Frequency: 1000 Hz
- Watchdog Timeout: 0.5 seconds
- Acceleration Rate: 5% per update

Navigation

- Search Rotation Speed: 30%
- Center Deadzone: 30 pixels
- Distance Deadzone: 10cm

Security Considerations

Network Security

- **CORS:** Configured for development (allows all origins)
- **No Authentication:** Currently no authentication required
- **Local Network:** Designed for local network use only

Safety Features

- **Watchdog Timer:** Automatic motor stop on timeout
- **Emergency Stop:** Immediate halt capability
- **Thermal Protection:** CPU temperature monitoring
- **Connection Timeout:** Automatic stop on disconnect

Future Enhancements

Potential Improvements

1. **Authentication:** User authentication and authorization
2. **SSL/TLS:** Encrypted communication
3. **Multi-Robot Support:** Control multiple robots
4. **Path Planning:** Advanced navigation algorithms
5. **Obstacle Avoidance:** Ultrasonic/LIDAR integration
6. **Mapping:** SLAM (Simultaneous Localization and Mapping)
7. **Machine Learning:** Custom object detection models
8. **Cloud Integration:** Remote access and data logging

Conclusion

VoyagerBot represents a comprehensive robotics platform combining real-time computer vision, autonomous navigation, and modern web technologies. The modular architecture enables easy extension and customization, while the responsive frontend provides an intuitive control interface accessible from any device.

The system demonstrates advanced features including: - Multi-threaded processing for optimal performance - Real-time video streaming with low latency - Intelligent object tracking and following - AI-powered voice control - Comprehensive telemetry and monitoring - Robust error handling and safety features

This documentation provides a complete overview of all implemented features and serves as a reference for developers, users, and stakeholders.

.

Document Version: 1.0

Last Updated: 2025

Project: VoyagerBot

Platform: Raspberry Pi 5 + Next.js Web Application